

Practice Lab

Submit all of your cpp files.

Submission format: **roll_problem.cpp**

Example: **61_A.cpp**

A. Strongly Connected Components (SCC)

Given a directed graph find its strongly connected components.

Input:

First line: N ($0 < N \leq 100000$), number of nodes.

Second line: M ($0 < M \leq 300000$), number of edges.

Next M lines, each: $U\ V$ ($0 \leq U, V < N$), defines an edge from U to V .

Output:

Strongly Connected Components.

Sample Input:

```
8
8
1 3
0 1
3 0
3 7
7 3
5 6
6 4
6 5
```

Sample Output:

```
6 5
4
2
1 7 3 0
```

B. Edge Classification

Given a directed graph, classify its edges. You should run DFS in increasing order. The graph will not contain multiple edges between a pair of nodes.

Input:

First line: N ($0 < N \leq 100$), number of nodes.

Second line: M ($0 < M \leq 4950$), number of edges.

Next M lines, each: $U\ V$ ($0 \leq U, V < N$), define an edge from U to V .

Output:

Edge classifications. Print edges in increasing order.

Sample Input:

```
6
7
0 1
1 2
2 0
2 3
0 3
4 5
5 1
```

Sample Output:

```
0 1 (T)
0 3 (F)
1 2 (T)
2 0 (B)
2 3 (T)
4 5 (T)
5 1 (C)
```

C. Articulation Points

Given an undirected graph find all of its articulation points.

Input:

First line: N ($0 < N \leq 100000$), number of nodes.

Second line: M ($0 < M \leq 300000$), number of edges.

Next M lines, each: $U\ V$ ($0 \leq U, V < N$), defines an edge between U and V .

Output:

List all the articulation points in increasing order.

Sample Input:

```
6
5
1 3
1 2
0 1
3 4
2 5
```

Sample Output:

```
1
2
3
```

D. Bridges

Given an undirected graph find all of its bridges.

Input:

First line: N ($0 < N \leq 100000$), number of nodes.

Second line: M ($0 < M \leq 300000$), number of edges.

Next M lines, each: $U\ V$ ($0 \leq U, V < N$), defines an edge between U and V .

Output:

List all the bridges in increasing order.

Sample Input:

```
6
6
1 3
1 2
2 3
0 1
3 4
5 0
```

Sample Output:

```
0 1
0 5
3 4
```

E. Topological Order

Given a directed acyclic graph (DAG) find a valid topological order.

Input:

First line: N ($0 < N \leq 100000$), number of nodes.

Second line: M ($0 < M \leq 300000$), number of edges.

Next M lines, each: $U\ V$ ($0 \leq U, V < N$), defines an edge from U to V .

Output:

Topological Order.

Sample Input:

```
6
5
1 3
1 2
0 1
3 4
2 5
```

Sample Output:

```
0
1
2
5
3
4
```